

1. PowerShell Basics

1.1. Profile (~/.bashrc equivalent)

<code>\$PROFILE</code>	Automatic variable holding the path of the profile script. It is run at every startup, like <code>~/bashrc</code> . Typically <code>...\Documents\PowerShell\Microsoft.PowerShell.profile.ps1</code> .
<code>\$PROFILE Select-Object *</code>	Show all profile paths (AllUsers / CurrentUser × AllHosts / CurrentHost).
<code>Test-Path \$PROFILE</code>	Check whether the profile file exists.
<code>New-Item -Type File \$PROFILE -Force</code>	Create the profile file (with parent dirs).
<code>notepad \$PROFILE</code>	Edit it. Put aliases, functions, prompt customization etc. here.
<code>. \$PROFILE</code>	Dot-source (reload) the profile in the current session, like <code>source ~/bashrc</code> .

Note: Running scripts may be disabled by default. Allow with:
`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

1.2. Everything is an object

Unlike bash, where commands pass *text* through pipes, PowerShell commands pass *objects*. Each object has properties (attributes) and methods. Eg: `Get-ChildItem` emits `FileInfo/DirectoryInfo` objects with properties like `Name`, `FullName`, `Length`, `LastWriteTime`, `Extension`.

<code>cmd Get-Member</code>	List the type, properties and methods of the objects a command outputs.
<code>(Get-Item f.txt).Length</code>	Access a property directly.
<code>"abc".ToUpper()</code>	Call a method.
<code>cmd Format-List *</code>	Show all properties with their values.

1.3. Script blocks, special variables, \$()

<code>{ ... }</code>	Script block: a chunk of code passed as an argument, like an anonymous function. Used with <code>Where-Object</code> , <code>ForEach-Object</code> , etc.
<code>\$_</code>	The current object coming through the pipeline (inside a script block). Alias: <code>\$PSItem</code> .
<code>\$_.Name</code>	Property of the current pipeline object. Eg: <code>\$_.FullName</code> , <code>\$_.Length</code> .
<code>\$(...)</code>	Subexpression: evaluate expression and use the result, mainly inside double-quoted strings. Eg: <code>"Size: \$(\$f.Length)"</code> . Like <code>\$()</code> in bash but evaluates PowerShell code, not a subshell.
<code>@(...)</code>	Array subexpression: force the result to be an array.
<code>\$env:PATH</code>	Environment variables live on the <code>env:</code> drive. Eg: <code>\$env:HOME</code> , <code>\$env:USERPROFILE</code> .
<code>\$null</code> , <code>\$true</code> , <code>\$false</code>	Automatic constants.
<code>\$?</code>	<code>\$true</code> if the last command succeeded.

1.4. Pipes and connecting commands

<code> </code>	Pipe. Passes <i>objects</i> (not text) to the next command. Eg: <code>Get-ChildItem Where-Object {\$_ .Length -gt 1MB}</code> .
<code>;</code>	Run commands in sequence on one line (same as bash).
<code>&&, </code>	Run next command only on success / failure (PowerShell 7+).
<code>`</code>	Backtick at end of line: continue command on next line (like <code>\</code> in bash). A line ending in <code> </code> also continues.
<code>>, >></code>	Redirect to file / append to file. <code>2></code> redirects errors; <code>*></code> redirects all streams.
<code>& "prog.exe"</code>	Call operator: run a command whose name is a string/variable.

2. Core Cmdlets

Cmdlets are named **Verb-Noun**. Case-insensitive. Discover with `Get-Command` and `Get-Help <cmdlet> -Examples`.

<code>Get-ChildItem</code>	List items in a location (like <code>ls</code>). Aliases: <code>gci</code> , <code>ls</code> , <code>dir</code> . Emits <code>FileInfo/DirectoryInfo</code> objects.
<code>-Recurse</code>	Recurse into subdirectories.
<code>-Filter *.txt</code>	Filter by wildcard pattern (fast, done by provider).
<code>-File</code> , <code>-Directory</code>	Only files / only directories.
<code>-Force</code>	Include hidden and system items (like <code>ls -a</code>).
<code>-Name</code>	Output only names (strings), not objects.
<code>-Depth n</code>	Limit recursion depth.
<code>Where-Object</code>	Filter objects by a condition. Alias: <code>where</code> , <code>?</code> . <code>gci Where-Object {\$_ .Length -gt 1MB}</code> Simplified syntax: <code>gci where Length -gt 1MB</code> .
<code>ForEach-Object</code>	Run a script block for each pipeline object (like a loop body). Alias: <code>foreach</code> , <code>%</code> . <code>gci ForEach-Object {\$_ .Name.ToUpper()}</code> <code>-Begin {} -Process {} -End {}</code> blocks like <code>awk</code> 's BEGIN/END.
<code>Select-Object</code>	Select properties of objects, or a subset of objects. Alias: <code>select</code> .
<code>-Property a,b</code>	Keep only listed properties. Eg: <code>gci select Name,Length</code> .
<code>-First n</code> , <code>-Last n</code>	Like <code>head -n</code> / <code>tail -n</code> .
<code>-Unique</code>	Drop duplicates (like <code>uniq</code>).
<code>-ExpandProperty x</code>	Output the raw property values instead of objects with one property.
<code>Select-String</code>	Search text in files/strings with regex (like <code>grep</code>). Alias: <code>sls</code> . <code>Select-String "pattern" *.log</code> Regex pattern (default parameter).
<code>-Pattern</code>	Literal match, no regex (like <code>grep -F</code>).
<code>-SimpleMatch</code>	Match is case-insensitive by default.
<code>-CaseSensitive</code>	Non-matching lines (like <code>grep -v</code>).
<code>-NotMatch</code>	Lines of context before,after (like <code>grep -B1 -A2</code>).
<code>-Context 1,2</code>	Output objects have <code>Line</code> , <code>LineNumber</code> , <code>Filename</code> , <code>Matches</code> properties.

Get-Content	Read a file and output it line by line (like cat). Aliases: gc, cat, type. Each line is a string object, so the result can be indexed: (gc f.txt)[0].
-TotalCount n	First n lines (like head -n).
-Tail n	Last n lines (like tail -n).
-Wait	Keep watching the file (like tail -f).
-Raw	Whole file as one single string, not an array of lines.
Write-Output	Write objects to the pipeline / stdout (like echo). Alias: write, echo. Usually implicit: a bare expression is output automatically.
Write-Host	Print directly to console (colors possible), bypassing the pipeline.
Out-File	Send output to a file. Eg: gci Out-File list.txt. > is shorthand for it.
-Append	Append instead of overwrite (>>).
-Encoding utf8	Set encoding.
-NoClobber	Do not overwrite an existing file.
New-Item	Create a file/directory/link. Alias: ni. New-Item f.txt (file by default) New-Item -ItemType Directory d
-Force	Create intermediate directories; overwrite existing.
-Value "text"	Create the file with initial content.
Remove-Item	Delete items (like rm). Aliases: ri, rm, del.
-Recurse	Delete directories with contents (rm -r).
-Force	Also hidden/read-only items (rm -f).
-WhatIf	Dry run: show what would be deleted. Works on most cmdlets.
-Confirm	Prompt before each deletion.
Sort-Object	Sort objects by property value (like sort). Alias: sort. gci Sort-Object Length
-Descending	Reverse order (like sort -r).
-Unique	Drop duplicates after sorting (like sort -u).
-Property a,b	Sort by a, break ties by b. Numbers sort numerically, not as text.
Measure-Object	Compute statistics over pipeline objects (like wc). Alias: measure. Default: count only. gci measure Length -Sum -Average
-Sum, -Average	Sum / mean of a numeric property.
-Minimum, -Maximum	Min / max of a property.
-Line -Word -Character	Count lines/words/chars of text (like wc): gc f measure -Line. Output is an object: get numbers via (...).Sum, (...).Lines, etc.

3. Operators

3.1. Comparison operators

Bash-style ==, > etc. are *not* comparison operators (> redirects!). String comparisons are case-insensitive by default; prefix c for case-sensitive (-ceq, -clike, -cmatch, ...), i for explicit case-insensitive (-ieq, ...).

-eq, -ne	Equal, not equal. Eg: 5 -eq 5, \$_.Name -ne "tmp".
-gt, -ge	Greater than, greater or equal.
-lt, -le	Less than, less or equal.
-in, -notin	Value contained in a collection: 3 -in 1,2,3.
-contains	Collection contains value: 1,2,3 -contains 3.
-and, -or, -not, !	Logical operators. Eg: (\$a -gt 1) -and (\$a -lt 9).

When applied to a collection, a comparison *filters* it: 1,2,3,4 -gt 2 gives 3,4.

3.2. String matching

-like	Wildcard match: * any chars, ? one char, [a-z] range. Eg: \$_.Name -like "*.txt".
-notlike	Negated wildcard match.
-match	Regex match: "abc123" -match "\d+". Captures go to the automatic \$Matches hashtable (\$Matches[0], \$Matches[1], ...).
-notmatch	Negated regex match.
-replace	Regex replace (like sed): "a-b" -replace "-", ":". Supports groups: -replace '(\w+)\@', '\$1 AT '.
-split	Split by regex: "a,b,c" -split ",", "
-join	Join collection: 1,2,3 -join "-" gives 1-2-3.

4. File attributes & permissions

Windows has no `rx` mode bits. Instead: *attributes* (ReadOnly, Hidden, System, Archive) and *ACLs* (access control lists) for per-user permissions.

4.1. attrib (cmd tool, chmod-like)

attrib file	Show attributes of a file.
attrib +r file	Set read-only. -r clears it (closest to chmod -w/+w).
attrib +h file	Hide file. -h unhides.
+s, -s	Set/clear system attribute.
+a, -a	Set/clear archive attribute.
/s /d	Apply recursively (/s), include directories (/d).

4.2. PowerShell native

(Get-Item f).Attributes	Read attributes as a property.
Set-ItemProperty f -Name IsReadOnly -Value \$true	Make read-only.
Get-Acl f	Show owner and access rules (like ls -l for permissions).
Get-Acl f Format-List	Readable full listing.
Set-Acl	Apply a (modified) ACL to a file.

4.3. icacls (ACLs, chmod/chown-like)

icacls file	Show permissions.
icacls f /grant user:F	Grant rights: F full, M modify, RX read+execute, R read, W write.
icacls f /deny user:W	Deny writing.
icacls f /remove user	Remove user's entries.
/t	Recurse into subdirectories.

Note: In PowerShell 7 on Linux, `chmod` etc. work as usual.

5. Bash equivalents

Many bash names exist as *aliases* of cmdlets (list them with `Get-Alias`), but flags differ: `ls -la` does *not* work; use the cmdlet's parameters.

cd	Set-Location (aliases: cd, sl). cd - works in PS 6+. Get-Location (pwd) shows the current dir. Push-Location/Pop-Location = pushd/popd.
mkdir	New-Item -ItemType Directory d or just mkdir d (creates parents automatically, like mkdir -p).
touch	No direct equivalent. Create: New-Item f.txt. Update timestamp: (Get-Item f).LastWriteTime = Get-Date.
tree	tree (cmd tool; /f to also list files). Or Get-ChildItem -Recurse.
cat	Get-Content f (aliases: cat, gc). -TotalCount n = head, -Tail n = tail, -Wait = tail -f. Set content: Set-Content.
xargs	Not needed: the pipeline already passes objects. gci *.log Remove-Item. For per-item commands use ForEach-Object { cmd \$_ }.
sed	-replace operator: (Get-Content f) -replace 'old','new' Set-Content f.
grep	Select-String "pattern" f. On command output: cmd Select-String "x" (text) or better, filter objects: cmd where Name -match "x".
cp, mv	Copy-Item (-Recurse for dirs), Move-Item (also renames, like mv).
rm	Remove-Item (see above).
ls	Get-ChildItem (see above).
echo	Write-Output.
which	Get-Command name.
man	Get-Help cmdlet -Examples / -Full.
history	Get-History; r n re-runs entry n.
wc -l	(Get-Content f Measure-Object -Line).Lines.
sort, uniq	Sort-Object (-Unique, -Descending), Group-Object for counting like uniq -c.
less	cmd Out-Host -Paging or more.
find	Get-ChildItem -Recurse -Filter "*.txt".
ln -s	New-Item -ItemType SymbolicLink -Path link -Target file.
df, du	Get-PSDrive; gci -Recurse Measure-Object Length -Sum.